5-Week RAG + Agents + MCP Professional Learning Plan

Comprehensive Step-by-Step Guide for Building Production-Ready AI Systems

---

Course Overview

This intensive 5-week program is designed to take you from foundational AI concepts to building production-ready Retrieval Augmented Generation (RAG) systems, AI agents, and Model Context Protocol (MCP) integrations. Each week builds upon the previous, culminating in a sophisticated AI application that combines all three technologies.

Prerequisites

- Basic programming experience (Python recommended)
- Familiarity with cloud platforms (Azure experience helpful)
- Understanding of REST APIs and web development concepts
- Access to Azure subscription (free tier available)

Learning Objectives

By the end of this program, you will be able to:

- Build and deploy sophisticated RAG systems using Azure services
- Develop multi-agent AI systems with advanced capabilities
- Implement Model Context Protocol for standardized AI integrations
- Create production-ready AI applications with proper monitoring and evaluation

---

Week 1 – Foundations & Azure Ecosystem Mastery

Day 1: AI Fundamentals & Azure Cloud Setup

Learning Resources:

- AI for Everyone - DeepLearning.AI - Complete Module 1 (Introduction to AI)
- Coursera AI for Everyone - Andrew Ng's foundational course

Detailed Implementation Steps:

1. **Azure Account Creation & Configuration**
    - Navigate to Azure Portal

- Create new Azure Free Account with $200 credit
- Complete identity verification process
- Set up billing alerts to monitor usage

2. **Resource Group Setup**
   - Access Azure Portal → Search "Resource groups"
   - Click "+ Create" → Name: "AI-Lab-RG"
   - Region: East US (for OpenAI compatibility)
   - Apply resource tags: Environment=Learning, Project=RAG-Agents
3. **Azure Navigation Mastery**
   - Navigate All services → AI + Machine Learning section
   - Bookmark frequently used services for quick access
   - Explore the free tier service limits and quotas
   - Review Azure pricing calculator for future reference
4. **Storage Infrastructure Setup**
   - Create Storage Account: "ailearningstorage[uniqueid]"
   - Performance: Standard, Replication: LRS
   - Enable Blob service with hot access tier
   - Create containers: "datasets", "models", "outputs"
   - Configure access levels and permissions

Detailed Output Expectations:

- Comprehensive Azure Environment: Fully configured Azure free tier subscription with resource group, storage account, and organized container structure ready for AI workloads

---

Day 2: OpenAI Integration & First AI Interactions

Learning Resources:

- AI for Everyone Modules 2-3
- Azure OpenAI Service Documentation
- OpenAI API Fundamentals

Detailed Implementation Steps:

1. **Azure OpenAI Service Deployment**
   - Navigate to Azure Portal → Create a resource
   - Search "Azure OpenAI" → Select → Create

- Resource details: Region=East US, Pricing tier=Standard S0
- Configure network access and managed identity
- Wait for deployment completion (5-10 minutes)

2. **Model Deployment Configuration**
   - Access Azure OpenAI Studio via resource overview
   - Navigate to Deployments → Create new deployment
   - Model: gpt-4 (latest version available)
   - Deployment name: "gpt-4-main"
   - Configure rate limits: 10K TPM, 100 RPM
3. **Interactive Testing Environment**
   - Open Chat Playground in Azure OpenAI Studio
   - Configure system message: "You are a helpful AI assistant"
   - Test with prompt: "Explain the fundamentals of artificial intelligence in simple terms"
   - Experiment with temperature settings (0.1, 0.5, 0.9)
   - Save conversation history and responses
4. **API Key Management**
   - Generate API keys in Keys and Endpoint section
   - Store securely in Azure Key Vault or local environment file
   - Test API connectivity using curl or Postman
   - Document endpoint URL and authentication method

Detailed Output Expectations:

- Functional OpenAI Integration: Working Azure OpenAI deployment with GPT-4, successful API authentication, documented responses to various prompts, and secure API key management system

---

Day 3: Transformer Architecture Deep Dive

Learning Resources:

- How Transformers Work - DataCamp
- Transformers Explained - Towards Data Science
- The Illustrated Transformer
- HuggingFace Transformers Documentation

Detailed Implementation Steps:

1. **Development Environment Setup**
   - Install Python 3.9+ and create virtual environment
   - Execute: `pip install jupyter notebook transformers torch`
   - Launch Jupyter: `jupyter notebook`
   - Create new notebook: "Transformer_Exploration.ipynb"
2. **HuggingFace Integration Exploration**
   - Install dependencies: `pip install transformers datasets tokenizers`
   - Import libraries and test basic functionality
   - Test various transformer tasks including fill-mask, text generation, and sentiment analysis
3. **Interactive Transformer Experiments**
   - Run fill-mask: `fill_mask("Artificial intelligence is [MASK] technology.")`
   - Test text generation with different prompts
   - Analyze tokenization process with various texts
   - Experiment with attention mechanisms visualization
4. **Azure ML Workspace Integration**
   - Create Azure Machine Learning workspace
   - Upload notebook to Azure ML Studio
   - Run experiments in cloud compute instances
   - Save outputs and model artifacts

Detailed Output Expectations:

- Transformer Implementation Mastery: Jupyter notebook with working transformer examples, successful text generation and analysis, uploaded cloud-based experiments, and documented understanding of attention mechanisms and model architectures

---

Day 4: Advanced Prompt Engineering & Function Development

Learning Resources:

- Anthropic Prompt Engineering Guide
- System Prompts Repository
- Azure Functions Documentation

Detailed Implementation Steps:

1. **Comprehensive Prompt Engineering Practice**
   - Create structured prompt templates for:

    - Summarization: "Summarize the following text in exactly 3 bullet points..."
    - Child-friendly explanations: "Explain [concept] as if to a 10-year-old..."
    - Pros/cons analysis: "Analyze [topic] providing 3 pros and 3 cons with reasoning..."
    - FAQ generation: "Create 5 frequently asked questions about [topic]..."
    - Technical documentation: "Write API documentation for [function]..."
  - Test each prompt with multiple examples
  - Document best practices and optimization techniques
2. **Azure Functions Development Environment**
  - Install Azure Functions Core Tools: `npm install -g azure-functions-core-tools@4`
  - Install VS Code with Azure Functions extension
  - Initialize new function project: `func init AIPromptBot --python`
  - Create HTTP trigger: `func new --name PromptProcessor --template "HTTP trigger"`
3. **AI-Powered Function Implementation**
  - Create HTTP trigger function that processes incoming prompts
  - Configure OpenAI client integration
  - Process requests and return structured AI responses
  - Include comprehensive error handling and logging
4. **Deployment and Testing**
  - Deploy function: `func azure functionapp publish <app_name>`
  - Test endpoints with various prompt types
  - Implement request/response logging
  - Set up monitoring and alerts

Detailed Output Expectations:

- Production-Ready Prompt Bot: Deployed Azure Function with 5+ optimized prompt templates, comprehensive error handling, logging system, and documented API endpoints that can process various prompt types and return structured AI responses

---

Day 5: Git Fundamentals & Version Control Mastery

Learning Resources:

- Git Official Documentation
- Pro Git Book (Free)
- GitHub Git Handbook

- Atlassian Git Tutorials
- Interactive Git Learning

Detailed Implementation Steps:

1. **Git Installation and Configuration**
   - Download and install Git from git-scm.com
   - Configure global user settings: `git config --global user.name "Your Name"`
   - Set up email: `git config --global user.email "your.email@example.com"`
   - Configure default editor and line endings
   - Set up SSH keys for secure authentication
   - Verify installation with `git --version`
2. **Repository Fundamentals**
   - Initialize new repository: `git init`
   - Clone existing repositories: `git clone <url>`
   - Understand repository structure (.git folder, working directory, staging area)
   - Create and manage .gitignore files for different project types
   - Practice with local repository creation and basic file operations
3. **Core Git Workflow Mastery**
   - Master the three-stage workflow: Working Directory → Staging Area → Repository
   - Practice file tracking: `git add`, `git commit`, `git status`
   - Understand staging area concept and selective staging
   - Learn commit message best practices and conventions
   - Practice viewing history with `git log`, `git show`, `git diff`
   - Understand HEAD, branches, and commit references
4. **Branching and Merging Excellence**
   - Create and switch branches: `git branch`, `git checkout`, `git switch`
   - Understand branch naming conventions (feature/, bugfix/, hotfix/)
   - Practice merging strategies: fast-forward, three-way merge, rebase
   - Resolve merge conflicts manually and with tools
   - Learn branch management: delete, rename, track remote branches
   - Practice advanced branching workflows (Git Flow, GitHub Flow)
5. **Remote Repository Management**
   - Connect to remote repositories: `git remote add origin <url>`

- Push and pull changes: `git push`, `git pull`, `git fetch`
- Understand origin, upstream, and multiple remotes
- Practice forking and contributing to open source projects
- Set up branch tracking and default push behavior
- Handle authentication with personal access tokens

6. **Advanced Git Operations**
   - Interactive staging: `git add -i`, `git add -p`
   - Commit amending and history modification: `git commit --amend`
   - Stashing work in progress: `git stash`, `git stash pop`
   - Cherry-picking commits: `git cherry-pick`
   - Reset and revert operations: `git reset`, `git revert`
   - Rebasing for clean history: `git rebase -i`
7. **GitHub Integration and Collaboration**
   - Create GitHub account and repositories
   - Understand public vs private repositories
   - Practice pull request creation and review process
   - Learn issue tracking and project management features
   - Set up GitHub Actions for basic CI/CD
   - Practice collaborative workflows with forks and pull requests
8. **Git Best Practices and Troubleshooting**
   - Establish commit message conventions and standards
   - Learn when to use different merge strategies
   - Practice recovering from common mistakes
   - Understand git reflog for recovery operations
   - Set up aliases for frequently used commands
   - Create and manage multiple SSH keys for different accounts
9. **Practical Project Setup**
   - Create repository structure for AI/ML projects
   - Set up appropriate .gitignore for Python, Node.js, and data files
   - Practice organizing code, documentation, and configuration files
   - Implement branching strategy for future week projects
   - Set up repository templates for consistent project structure

Detailed Output Expectations:

- Git Mastery Foundation: Complete understanding of Git fundamentals with hands-on practice, configured development environment, GitHub repository setup, demonstrated branching and merging skills, and established best practices for version control in AI/ML projects

---

Week 2 – RAG Fundamentals & Application Development & Document Intelligence

Day 6: Vector Databases & Document Indexing Foundation

Learning Resources:

- Vector Databases Fundamentals
- Azure Cognitive Search Documentation
- Document Chunking Strategies

Detailed Implementation Steps:

1. **Azure Cognitive Search Service Setup**
   - Create Azure Cognitive Search resource
   - Choose appropriate pricing tier (Basic for learning)
   - Configure search units and replicas
   - Set up role-based access control (RBAC)
   - Create API keys for programmatic access
2. **Document Upload and Storage Configuration**
   - Upload sample PDF to Blob Storage "datasets" container
   - Configure blob storage connection string
   - Set up container access policies
   - Create folder structure: /documents/pdf/, /documents/text/
   - Implement automated document processing pipeline
3. **Search Index Creation and Configuration**
   - Use Import data wizard in Cognitive Search
   - Configure data source pointing to blob storage
   - Define index schema with fields:
     - content (searchable, retrievable)
     - metadata_title (searchable, filterable)

    - metadata_creation_date (filterable, sortable)
    - chunk_id (retrievable, filterable)
  - Configure cognitive skills for text extraction
4. **Search Testing and Optimization**
  - Test various search queries using Search Explorer
  - Analyze search results relevance and ranking
  - Optimize indexing settings for performance
  - Document search latency and accuracy metrics

Detailed Output Expectations:

- Functional Document Search System: Azure Cognitive Search service with indexed PDF documents, optimized search schema, tested query performance, and documented search capabilities with response times under 100ms

---

Day 7: Advanced Text Processing & Embedding Generation

Learning Resources:

- Context Windows and Token Management
- LangChain Document Processing
- Azure OpenAI Embeddings Guide
- OpenAI Embeddings Documentation
- Sentence Transformers Documentation
- Understanding Vector Embeddings
- Embedding Evaluation Methods

Detailed Implementation Steps:

1. **Advanced Text Processing Pipeline**
  - Install dependencies: `pip install langchain azure-search-documents openai tiktoken`
  - Implement intelligent chunking strategy:
    - Semantic chunking based on sentence boundaries
    - Overlap management (50-100 tokens)
    - Metadata preservation throughout chunking
    - Special handling for tables, lists, and code blocks
2. **Embedding Generation System**

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter
from azure.search.documents import SearchClient
import openai

def process_document(file_path):
    # Load and split document
    # Generate embeddings for each chunk
    # Store in Azure Cognitive Search
    # Return processing statistics
```

3. **Chunk Optimization and Analysis**
   - Experiment with different chunk sizes (256, 512, 1024 tokens)
   - Analyze semantic coherence across chunk boundaries
   - Implement quality scoring for chunks
   - Create visualization of chunk distribution and overlap
4. **Vector Index Population & Performance Testing**
   - **Multi-Model Embedding Comparison:**
     - Generate embeddings using Azure OpenAI text-embedding-ada-002 (1536d)
     - Test Sentence Transformers all-MiniLM-L6-v2 (384d) for cost comparison
     - Experiment with all-mpnet-base-v2 (768d) for balanced performance
     - Compare retrieval accuracy across different dimensional models
   - **Embedding Storage and Indexing:**
     - Store vectors in Cognitive Search with hybrid configuration
     - Implement batch processing for large document collections
     - Create embedding metadata with model type and dimension info
     - Set up vector similarity search with configurable distance metrics
   - **Performance Benchmarking:**
     - Measure embedding generation time vs model complexity
     - Compare storage requirements across different dimensions
     - Test query response times for different embedding sizes
     - Analyze retrieval accuracy using precision@k and recall@k metrics
   - **Cost Analysis and Optimization:**
     - Calculate embedding generation costs for different models
     - Analyze storage costs based on vector dimensions
     - Create cost-performance optimization recommendations

    - Implement error handling and retry logic for API calls
    - Monitor processing costs and performance metrics

Detailed Output Expectations:

- Advanced Text Processing & Embedding System: Comprehensive pipeline comparing multiple embedding models (384-3072 dimensions), automated processing of diverse PDFs with optimized chunking strategies, performance benchmarking across different dimensional approaches, and cost-benefit analysis with recommendations for production deployment

---

Day 8: RAG System Implementation & Integration

Learning Resources:

- RAG Course - DeepLearning.AI
- Building RAG Applications
- LangChain RAG Tutorial

Detailed Implementation Steps:

1. **RAG Pipeline Architecture Design**
    - Design modular RAG system with components:
        - Query processor and intent classification
        - Retrieval engine with ranking algorithms
        - Context synthesis and prompt construction
        - Response generation with source attribution
        - Answer quality evaluation and filtering
2. **Retrieval System Implementation**

```python
```

```python
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.llms import AzureOpenAI

class AdvancedRAGSystem:
    def __init__(self):
        self.retriever = self._setup_retriever()
        self.llm = self._setup_llm()
        self.qa_chain = self._create_qa_chain()

    def query(self, question, num_results=5):
        # Implement advanced retrieval logic
        # Process and rank results
        # Generate contextualized response
        return response_with_sources
```

3. **Context Management and Prompt Engineering**
   - Create dynamic prompt templates based on query type
   - Implement context windowing for long documents
   - Add source attribution and confidence scoring
   - Handle edge cases: no results, conflicting information
   - Implement conversation memory for multi-turn interactions
4. **Testing and Validation Framework**
   - Create comprehensive test dataset with Q&A pairs
   - Implement automated evaluation metrics
   - Test query performance across different document types
   - Validate answer accuracy and source attribution
   - Document system limitations and improvement areas

Detailed Output Expectations:

- Production-Grade RAG System: Complete RAG application with intelligent retrieval, contextual answer generation, source attribution, comprehensive testing framework, and documented performance metrics achieving >85% answer accuracy

---

Day 9: Multi-Document RAG & Persona Prompt Management

Learning Resources:

- PIKE-RAG: Advanced Retrieval Techniques

- Awesome RAG Repository
- Multi-Document Question Answering
- Advanced Prompt Engineering Patterns

Detailed Implementation Steps:

1. **Multi-Document Processing Pipeline**
   - Upload diverse document types: PDFs, Word docs, PowerPoint
   - Implement document type detection and specialized parsers
   - Create unified metadata schema across document types
   - Establish document versioning and update mechanisms
2. **Advanced Metadata Management**
   - Create unified metadata schema across document types
   - Include document ID, title, author, creation date, document type, topic tags, and security level
   - Implement consistent metadata extraction for all document formats
3. **Intelligent Query Routing and Filtering**
   - Implement query classification for document type routing
   - Add temporal filtering for date-based queries
   - Create author and topic-based filtering systems
   - Implement security-aware document access controls
4. **Cross-Document Synthesis**
   - Develop algorithms for combining information across documents
   - Implement conflict resolution for contradictory information
   - Create document relationship mapping
   - Add citation and reference tracking between documents
5. **Performance Optimization**
   - Implement caching layer for frequent queries
   - Add index optimization for multi-field searches
   - Create query performance monitoring dashboard
   - Optimize for cost-effective operations at scale

Detailed Output Expectations:

- Enterprise-Ready Multi-Document RAG with Persona Management: Sophisticated system handling diverse document types with intelligent routing, cross-document synthesis, role-

based prompt management for HR/Legal/Support roles, performance optimization, and comprehensive monitoring achieving sub-second response times for complex queries

---

Day 10: Interactive Chatbot Development & Week Review

Learning Resources:

- FastAPI Documentation
- Streamlit Documentation
- Azure Container Apps

Detailed Implementation Steps:

1. **RESTful API Development**
   - Create FastAPI application with CORS middleware
   - Implement endpoints for querying (/ask) and document management (/documents)
   - Add structured request/response handling
   - Include comprehensive error handling and validation
2. **Interactive Streamlit Application Development**
   - Install dependencies: `pip install streamlit openai pandas plotly`
   - Create comprehensive chatbot with multi-turn conversation handling
   - Implement session state management and chat history export
   - Add file upload capabilities and response streaming
   - Include sidebar with model parameters and prompt template selector
3. **Enhanced UI/UX Features**
   - Add chat history visualization and response time metrics
   - Create user feedback collection system
   - Implement persona-based chat modes (HR, Legal, L1/L2 Support)
   - Add analytics dashboard for usage tracking
   - Include FAQ system with semantic search capabilities
4. **Deployment and Integration**
   - Create Docker containers for API and frontend
   - Deploy to Azure Web Apps with custom domain
   - Configure GitHub Actions for CI/CD pipeline
   - Set up monitoring and health check endpoints
   - Implement load balancing and auto-scaling

5. **Comprehensive System Testing**
    - Create end-to-end test suite covering all functionality
    - Perform load testing with concurrent users
    - Validate persona-based prompt management
    - Test RAG integration across multiple document types
    - Document deployment procedures and user guides
6. **Professional Documentation and Demo**
    - Create API documentation with Swagger/OpenAPI
    - Record comprehensive demo video showcasing all features
    - Prepare technical presentation for stakeholders
    - Document architecture decisions and system capabilities
    - Create user manual and troubleshooting guide

Detailed Output Expectations:

- Complete Interactive RAG Application: Full-featured system with professional UI, comprehensive API, persona-based chatbot, load testing results, complete documentation, and demonstrated scalability to handle multiple concurrent users with integrated RAG capabilities

---

Week 3 – Advanced RAG Optimization & Performance

Day 11: RAG Evaluation & Comparison Analysis

Learning Resources:

- RAG Evaluation Techniques
- Advanced RAG Strategies Repository
- RAGAS Evaluation Framework

Detailed Implementation Steps:

1. **Chunking Strategy Evaluation Framework**
    - Implement multiple chunking strategies:
        - Fixed-size chunking (512, 1024, 2048 tokens)
        - Semantic chunking based on sentence boundaries
        - Paragraph-aware chunking with context preservation
        - Sliding window with variable overlap (50, 100 tokens)
2. **Comprehensive Testing Infrastructure**

- Create RAGEvaluator class to test different chunking strategies
- Load test dataset and ground truth for comparison
- Implement evaluation methods for each chunking strategy
- Calculate accuracy, relevance, and retrieval metrics for comparison

3. **Multi-Dimensional Performance Analysis**
   - Measure retrieval accuracy using precision@k and recall@k
   - Evaluate answer quality using BLEU, ROUGE, and semantic similarity
   - Analyze response latency and computational costs
   - Test context preservation across chunk boundaries
   - Compare factual accuracy and hallucination rates
4. **Statistical Analysis and Reporting**
   - Create detailed comparison tables with statistical significance testing
   - Generate visualizations showing performance trade-offs
   - Implement A/B testing framework for ongoing optimization
   - Document optimal configurations for different use cases
   - Create recommendations for strategy selection

Detailed Output Expectations:

- Comprehensive RAG Evaluation Report: Detailed analysis of 4+ chunking strategies with statistical validation, performance benchmarks, cost analysis, visualization dashboards, and evidence-based recommendations for optimal configurations

---

Day 12: Enterprise Data Ingestion Pipeline & Advanced Processing

Learning Resources:

- Azure ML Pipelines Documentation
- Docling Documentation
- Advanced Document Processing
- Web Scraping Best Practices

Detailed Implementation Steps:

1. **Multi-Source Data Ingestion Architecture**
   - Design ingestion pipeline for diverse enterprise data sources:
     - Website crawling from vendor sources (AWS Documentation, Amazon Developer Guides, Cisco Technical Docs)

    - Application log processing from various systems
    - ServiceNow integration for CRs, tickets, and service requests
    - Knowledge base article ingestion
    - Email and communication data processing
  - Create unified data connector framework
  - Implement source-specific authentication and access controls
2. **Advanced Document Processing with Docling**
  - Install Docling and configure multi-format processing: `pip install docling`
  - Implement document conversion for PDF, DOCX, PPTX, HTML formats
  - Create structured content extraction with table and figure processing
  - Add document quality assessment and validation
  - Implement cross-reference and citation tracking
3. **Intelligent Metadata Organization and Labeling**
  - Create comprehensive metadata schema:
    - Source system identification (ServiceNow, AWS Docs, Application Logs)
    - Document classification (Technical, Policy, Incident, Knowledge)
    - Urgency/Priority levels for tickets and service requests
    - Technical domain tags (Infrastructure, Security, Development)
    - Date/time stamps and version tracking
    - Access control and sensitivity levels
  - Implement automated metadata extraction and enrichment
  - Create metadata validation and quality assurance processes
4. **Advanced Chunking and Processing Methods**
  - Implement domain-aware chunking strategies:
    - Technical documentation: section-based chunking
    - ServiceNow tickets: field-based structured extraction
    - Application logs: pattern-based parsing and grouping
    - Knowledge articles: semantic topic-based chunking
  - Create intelligent overlap management for different content types
  - Add special processing for code snippets, configuration files, and structured data
  - Implement content deduplication and similarity detection
5. **Pipeline Orchestration and Quality Management**
  - Create automated scheduling for different data sources

- Implement data validation and quality scoring
- Add error handling and retry mechanisms for failed ingestions
- Create monitoring dashboards for ingestion health
- Set up alerting for data quality issues or ingestion failures

Detailed Output Expectations:

- Enterprise-Grade Ingestion Pipeline: Comprehensive automated pipeline processing diverse data sources (websites, logs, ServiceNow, KB articles) with Docling integration, intelligent metadata labeling, advanced chunking methods, quality management, and monitoring dashboards

---

Day 13: Hybrid Search Implementation & Optimization

Learning Resources:

- Hybrid Search Theory and Practice
- Azure Cognitive Search Hybrid Documentation
- BM25 and Dense Retrieval Comparison

Detailed Implementation Steps:

1. **Hybrid Search Architecture Setup**
   - Configure Azure Cognitive Search with vector and keyword capabilities
   - Implement BM25 keyword search with custom scoring profiles
   - Set up dense vector search using text-embedding-ada-002
   - Create hybrid ranking algorithm combining both approaches
2. **Advanced Search Implementation**
   - Create HybridSearchEngine class combining vector and keyword search
   - Implement parallel vector and keyword search execution
   - Add rank fusion algorithm for combining results
   - Include configurable weighting parameters for different search types
3. **Search Algorithm Optimization**
   - Implement Reciprocal Rank Fusion (RRF) algorithm
   - Test different weighting strategies (50/50, 70/30, 30/70)
   - Optimize search parameters: top-k, similarity thresholds
   - Create query type classification for adaptive search strategies
   - Implement query expansion and suggestion algorithms

4. **Comprehensive Search Evaluation**
    - Create diverse test query dataset (factual, conceptual, multi-hop)
    - Measure search performance: precision, recall, MRR, NDCG
    - Compare hybrid vs. pure vector vs. pure keyword approaches
    - Analyze query types that benefit most from each approach
    - Document optimal configurations for different use cases

Detailed Output Expectations:

- Optimized Hybrid Search System: Production-ready search engine combining vector and keyword search with intelligent rank fusion, comprehensive performance evaluation, and documented optimal configurations achieving >90% user satisfaction on diverse query types

---

Day 14: Performance Optimization & Caching Strategies

Learning Resources:

- Redis Caching Patterns
- Azure Application Insights
- Database Optimization Techniques

Detailed Implementation Steps:

1. **Multi-Level Caching Architecture**
    - Deploy Azure Cache for Redis (Premium tier for persistence)
    - Implement application-level caching with TTL strategies
    - Create intelligent cache invalidation policies
    - Set up distributed caching for multi-instance deployments
2. **Advanced Caching Implementation**
    - Create IntelligentCacheManager with Redis integration
    - Implement get_or_compute pattern with TTL strategies
    - Add cache statistics and performance monitoring
    - Include intelligent cache key generation and invalidation policies
3. **Comprehensive Monitoring Implementation**
    - Configure Azure Application Insights with custom telemetry
    - Implement detailed performance counters and metrics
    - Create real-time dashboards for system health
    - Set up intelligent alerting based on performance thresholds

- Track business metrics: query satisfaction, response accuracy

4. **Performance Optimization Analysis**
   - Implement A/B testing for different optimization strategies
   - Measure impact of caching on response times and costs
   - Analyze query patterns for cache optimization
   - Create performance baselines and improvement targets
   - Document optimization techniques and their effectiveness

Detailed Output Expectations:

- High-Performance RAG System: Optimized system with multi-level caching achieving <500ms average response time, comprehensive monitoring dashboards, detailed performance analytics, and documented optimization strategies reducing operational costs by >30%

Day 15: Week Review & Performance Analysis

Learning Resources:

- System Architecture Best Practices
- Cost Optimization Strategies

Detailed Implementation Steps:

1. **Comprehensive System Deployment**
   - Deploy optimized RAG system to production environment
   - Configure auto-scaling based on demand patterns
   - Implement blue-green deployment strategy
2. **Cost Analysis and Optimization**
   - Collect detailed cost metrics across all Azure services:
     - Azure OpenAI API usage and token consumption
     - Azure Cognitive Search operations and storage costs
     - Compute costs for Azure ML pipelines
     - Storage costs for documents and embeddings
     - Redis cache and monitoring costs
3. **Performance vs Cost Trade-off Analysis**
   - Create CostPerformanceAnalyzer to evaluate different configurations
   - Test Basic, Optimized, and Premium configurations
   - Measure performance metrics and calculate costs for each configuration

- Generate recommendations based on cost-benefit analysis

4. **Executive Summary and Recommendations**
   - Create detailed technical report with architecture diagrams
   - Provide cost-benefit analysis for different deployment options
   - Document scalability limits and recommended scaling strategies
   - Create maintenance procedures and operational runbooks
   - Prepare executive presentation with business impact metrics
5. **Professional Review Preparation**
   - Organize comprehensive demo showcasing all capabilities
   - Prepare technical deep-dive presentation
   - Document lessons learned and improvement opportunities
   - Create transition plan for next week's agent development
   - Compile portfolio of week's achievements

Detailed Output Expectations:

- Enterprise-Grade RAG Solution: Complete production deployment with detailed cost analysis, performance benchmarks, executive summary, operational procedures, and professional presentation demonstrating ROI and technical excellence

---

Week 4 – AI Agents & Autonomous Systems

Day 16: AI Agent Fundamentals & Basic Implementation

Learning Resources:

- AI Agents Course - DeepLearning.AI
- LangChain Agents Documentation
- ReAct: Reasoning and Acting

Detailed Implementation Steps:

1. **Agent Architecture Foundation**
   - Install comprehensive agent framework: `pip install langchain langgraph langsmith`
   - Study agent design patterns: ReAct, Plan-and-Execute, Self-Correction
   - Implement basic agent components: memory, tools, reasoning loop
   - Create agent state management and execution tracking
2. **Mathematical Tool Integration**

- Create MathematicalAgent class with calculator and Python REPL tools
- Implement safe mathematical evaluation with error handling
- Add tools for basic calculations and complex Python code execution
- Include comprehensive testing for mathematical operations

3. **Agent Testing and Validation**
   - Create comprehensive test cases for mathematical operations
   - Test agent reasoning capabilities with complex problems
   - Validate tool selection and execution accuracy
   - Implement safety checks and error handling
   - Document agent decision-making process
4. **Agent Monitoring and Debugging**
   - Implement detailed logging for agent actions and decisions
   - Create debugging interface to trace agent reasoning
   - Set up performance metrics for tool usage efficiency
   - Add visualization for agent execution flow
   - Create agent behavior analysis tools

Detailed Output Expectations:

- Functional AI Agent System: Working agent with mathematical capabilities, comprehensive testing framework, detailed logging and monitoring, documented reasoning processes, and safety mechanisms ensuring reliable autonomous operation

---

Day 17: RAG-Enhanced Agent Development

Learning Resources:

- Microsoft AI Agents Documentation
- Agent-RAG Integration Patterns

Detailed Implementation Steps:

1. **Agent-RAG Integration Architecture**
   - Design hybrid system combining autonomous agents with RAG capabilities
   - Create intelligent tool selection mechanism for document queries
   - Implement context-aware decision making for information retrieval
   - Set up agent memory integration with document knowledge
2. **Advanced Agent-RAG Implementation**

- Create RAGAgent class using LangGraph StateGraph
- Implement tools for document search, summarization, and source comparison
- Add workflow creation and tool integration capabilities
- Include context-aware decision making for information retrieval

3. **Multi-Modal Query Processing**
   - Implement query classification for appropriate tool selection
   - Create complex query decomposition capabilities
   - Add cross-document synthesis and comparison features
   - Implement fact-checking and source verification
   - Create citation and reference management system
4. **Agent Performance Optimization**
   - Optimize tool selection algorithms for query types
   - Implement caching for frequently accessed documents
   - Create parallel processing for multi-step queries
   - Add intelligent retry mechanisms for failed operations
   - Implement cost optimization for API usage

Detailed Output Expectations:

- Sophisticated RAG-Agent System: Integrated system combining autonomous agents with advanced RAG capabilities, intelligent tool selection, multi-document reasoning, comprehensive testing, and optimized performance for complex information tasks

---

Day 18: Multi-Tool Agent Ecosystem

Learning Resources:

- 500 AI Agents Repository
- Tool Integration Patterns
- External API Integration

Detailed Implementation Steps:

1. **External API Integration Framework**
   - Set up weather API integration (OpenWeatherMap or similar)
   - Create secure API key management system
   - Implement rate limiting and error handling for external calls
   - Add response validation and error recovery mechanisms

2. **Multi-Tool Agent Development**
    - Create MultiToolAgent class integrating weather, RAG, calculator, web search, and calendar tools
    - Implement IntelligentToolSelector for automatic tool selection
    - Add parallel and sequential tool execution capabilities
    - Include result synthesis and response generation
3. **Complex Query Processing**
    - Implement query parsing for multi-step requests
    - Create dependency resolution for tool execution order
    - Add parallel execution capabilities for independent tasks
    - Implement result aggregation and synthesis algorithms
    - Create conflict resolution for contradictory information
4. **Agent Coordination and Workflow Management**
    - Design workflow orchestration for complex multi-step tasks
    - Implement agent state persistence across tool executions
    - Create checkpoint and rollback mechanisms for failed operations
    - Add comprehensive logging and audit trails
    - Implement performance monitoring and optimization

Detailed Output Expectations:

- Advanced Multi-Tool Agent: Sophisticated agent ecosystem integrating weather APIs, RAG systems, and multiple tools with intelligent coordination, parallel processing, comprehensive error handling, and documented workflow management capabilities

---

Day 19: Multi-Agent Collaboration Framework

Learning Resources:

- Multi-Agent Systems Design
- Agent Communication Protocols
- Collaborative AI Patterns

Detailed Implementation Steps:

1. **Multi-Agent Architecture Design**
    - Design specialized agent roles and responsibilities:
        - Research Agent: Information gathering and fact verification

    - Writer Agent: Content creation and editing
    - Quality Assurance Agent: Review and improvement suggestions
    - Coordinator Agent: Task distribution and workflow management

2. **Agent Communication Framework**
    - Create message types for task requests, completion, information sharing, and quality feedback
    - Implement AgentMessage dataclass with sender, receiver, message type, content, and timestamp
    - Design MultiAgentOrchestrator with message queue and workflow engine
    - Add task decomposition and coordination capabilities
3. **Collaborative Workflow Implementation**
    - Create handoff mechanisms between agents
    - Implement quality review and feedback loops
    - Add version control for collaborative content creation
    - Create conflict resolution mechanisms for disagreements
    - Implement progress tracking and milestone management
4. **Advanced Collaboration Features**
    - Add real-time collaboration monitoring dashboard
    - Implement agent performance evaluation and optimization
    - Create dynamic task redistribution based on agent availability
    - Add learning mechanisms for improved collaboration over time
    - Implement security and access control for sensitive tasks

Detailed Output Expectations:

- Production Multi-Agent System: Sophisticated collaborative agent framework with specialized roles, intelligent task coordination, quality assurance mechanisms, real-time monitoring, and documented collaboration protocols achieving >95% task completion accuracy

---

Day 20: Agent Deployment & Week Review

Learning Resources:

- Azure Container Apps for AI
- Microservices Architecture Patterns

Detailed Implementation Steps:

1. **Containerized Agent Deployment**
    - Create Docker containers for each specialized agent
    - Implement microservices architecture with service discovery
    - Set up load balancing and auto-scaling for agent services
    - Configure secure inter-service communication with API gateways
2. **Production Agent System Architecture**
    - Create Docker containers for each specialized agent
    - Configure base Python images with required dependencies
    - Set up proper working directories and file copying
    - Configure port exposure and startup commands for microservices
3. **Comprehensive Integration Testing**
    - Create end-to-end test scenarios for multi-agent workflows
    - Test system resilience under various conditions
    - Test agent coordination under high load conditions
    - Validate security and access control mechanisms
4. **Agent System Monitoring and Analytics**
    - Deploy comprehensive monitoring with Azure Monitor
    - Create custom dashboards for agent performance metrics
    - Implement intelligent alerting for system anomalies
    - Add business intelligence reporting for agent effectiveness
    - Create cost tracking and optimization recommendations
5. **Professional Documentation and Demo**
    - Create comprehensive system architecture documentation
    - Record detailed demo showcasing multi-agent collaboration
    - Prepare technical presentation for stakeholders
    - Document operational procedures and maintenance guides
    - Create user training materials and best practices guide

Detailed Output Expectations:

- Enterprise Multi-Agent Platform: Fully deployed, containerized multi-agent system with microservices architecture, comprehensive monitoring, professional documentation, and demonstrated capability to handle complex collaborative tasks at enterprise scale

---

Week 5 – Model Context Protocol & Final Integration

Day 21: MCP Fundamentals & Implementation

Learning Resources:

- Model Context Protocol Documentation
- MCP Course - Hugging Face
- MCP Quickstart Repository
- Anthropic MCP Course

Detailed Implementation Steps:

1. **MCP Development Environment Setup**
   - Clone MCP starter repository: `git clone https://github.com/modelcontextprotocol/quickstart`
   - Install Node.js dependencies: `npm install`
   - Set up TypeScript development environment
   - Configure MCP server development tools
2. **Basic MCP Server Implementation**
   - Create BasicMCPServer class using MCP SDK
   - Configure server with name, version, and capabilities
   - Set up tool handlers for listing and calling tools
   - Implement search_documents tool with query and limit parameters
3. **MCP Protocol Understanding and Testing**
   - Study MCP message types and communication patterns
   - Implement basic tools, resources, and prompts
   - Test MCP server with Claude Desktop integration
   - Create debugging and logging mechanisms for MCP communications
   - Validate protocol compliance and error handling
4. **Integration with Existing Systems**
   - Connect MCP server to previously built RAG system
   - Implement agent capabilities through MCP tools
   - Create resource endpoints for document access
   - Add prompt templates for common use cases
   - Test integration with multiple MCP clients

Detailed Output Expectations:

- Functional MCP Implementation: Working MCP server with tools, resources, and prompts, successful Claude Desktop integration, comprehensive testing framework, and documented protocol implementation

---

Day 22: Advanced MCP Integration & Orchestration

Learning Resources:

- Lastmile MCP Agent
- LangGraph Course
- MCP Best Practices

Detailed Implementation Steps:

1. **Advanced MCP Architecture Design**
   - Design MCP orchestration layer for complex workflows
   - Create intelligent routing between RAG and Agent systems
   - Implement context preservation across MCP interactions
   - Add session management and state persistence
2. **MCP Orchestrator Implementation**
   - Create MCPOrchestrator class managing multiple MCP servers
   - Implement intelligent routing engine for request distribution
   - Add distributed workflow execution capabilities
   - Include response synthesis from multiple MCP server results
3. **Container-Based MCP Deployment**
   - Create Azure Container Instance for MCP orchestrator
   - Implement service discovery for distributed MCP servers
   - Set up load balancing and health checking
   - Configure secure inter-service communication
   - Add comprehensive monitoring and logging
4. **MCP Integration Testing**
   - Create end-to-end test scenarios for complex workflows
   - Test integration with multiple AI clients (Claude, custom applications)
   - Validate protocol compliance across different scenarios
   - Test performance under various load conditions
   - Create automated regression testing suite

Detailed Output Expectations:

- Enterprise MCP Orchestration: Sophisticated MCP orchestration system deployed on Azure Container Instances with intelligent routing, comprehensive testing, performance monitoring, and demonstrated integration with multiple AI clients

---

Day 23: Advanced Document Processing with Docling

Learning Resources:

- Docling Documentation
- Advanced Document Processing
- Multi-Format Document Ingestion

Detailed Implementation Steps:

1. **Docling Integration Setup**
   - Install Docling and dependencies: `pip install docling`
   - Configure document processing pipeline
   - Set up support for multiple document formats (PDF, DOCX, PPTX, HTML)
   - Create document quality assessment tools
2. **Advanced Document Processing Pipeline**
   - Create AdvancedDocumentProcessor using Docling DocumentConverter
   - Implement document conversion to structured format
   - Add structured content extraction and quality assessment
   - Include enhanced embedding generation with metadata
3. **Enhanced RAG Integration**
   - Integrate Docling-processed documents into existing RAG pipeline
   - Implement document structure awareness in retrieval
   - Add table and figure processing capabilities
   - Create metadata-rich indexing with document structure information
   - Implement cross-reference and citation tracking
4. **Document Processing Analytics**
   - Create processing quality metrics and reporting
   - Implement document type classification and optimization
   - Add processing time and cost analysis
   - Create document improvement recommendations

- Build processing performance monitoring dashboard

Detailed Output Expectations:

- Advanced Document Processing System: Comprehensive document processing pipeline using Docling with multi-format support, quality assessment, structured content extraction, enhanced RAG integration, and detailed analytics dashboard

---

Day 24: Research Paper Analysis & Implementation

Learning Resources:

- Attention Is All You Need - Original Transformer Paper
- Recent RAG Advances
- LlamaIndex Improvements
- FreshLLMs Research

Detailed Implementation Steps:

1. **Research Paper Study and Analysis**
   - Conduct comprehensive study of assigned research papers
   - Create detailed summaries of key contributions and innovations
   - Identify practical applications for current systems
   - Document potential implementation strategies
   - Analyze computational and architectural requirements
2. **Key Insights Documentation**
   - Create detailed summaries for each research paper
   - Document key innovations like self-attention mechanisms and FreshLLMs techniques
   - Identify practical applications for current systems
   - Note implementation opportunities for enhanced query-document matching and content freshness
3. **Improvement Strategy Implementation**
   - Apply research insights to improve existing RAG retrieval algorithms
   - Implement attention-based document relevance scoring
   - Create dynamic content freshness detection system
   - Add advanced query expansion techniques based on recent research
   - Implement prototype features for next-generation capabilities
4. **Innovation Integration Testing**

- Test research-based improvements against baseline systems
- Measure performance improvements in real-world scenarios
- Create A/B testing framework for research-based features
- Document implementation challenges and solutions
- Create roadmap for further research integration

Detailed Output Expectations:

- Research-Enhanced System: Comprehensive analysis of cutting-edge research with practical implementations, measurable improvements to existing systems, documented innovation integration, and roadmap for future research applications

---

Day 25: Final Integration & Comprehensive Review

Learning Resources:

- Production AI System Design
- AI System Security

Detailed Implementation Steps:

1. **Final System Integration**
   - Integrate all components: RAG + Agents + MCP + Docling
   - Create unified API layer for all system capabilities
   - Implement comprehensive error handling and recovery
   - Add security layers and access controls
2. **Production-Ready Application Development**
   - Create UnifiedAISystem class integrating all components
   - Implement FastAPI application with HTTPBearer security
   - Add unified query processing endpoint with authentication
   - Include comprehensive error handling and response formatting
3. **Comprehensive Testing and Validation**
   - Create end-to-end test scenarios covering all integrated capabilities
   - Perform load testing with realistic usage patterns
   - Validate security and compliance requirements
   - Create user acceptance testing procedures
4. **Professional Documentation and Deployment**
   - Create comprehensive system architecture documentation

- Develop user manuals and API documentation
- Implement monitoring and alerting systems
- Deploy to production environment with observability

5. **Final Review and Presentation**
    - Create executive summary of achievements and capabilities
    - Prepare comprehensive technical demonstration
    - Document lessons learned and recommendations
    - Create business case and ROI analysis
    - Present final system to stakeholders with success metrics

Detailed Output Expectations:

- Complete Enterprise AI Platform: Fully integrated production system combining RAG, Agents, and MCP with comprehensive documentation, monitoring, security, and professional presentation demonstrating advanced AI capabilities and business value

---

Assessment

Weekly Review Structure

Each week concludes with a comprehensive review session including:

- **Technical Demonstration**: Working system showcase
- **Code Review**: Architecture and implementation quality assessment
- **Performance Analysis**: Metrics and optimization evaluation
- **Documentation Review**: Technical and user documentation quality
- **Presentation Skills**: Professional communication of technical concepts

---

Additional Resources

Essential Tools and Platforms

- **Azure Subscription**: Free Account Setup
- **Development Environment**: VS Code with Azure extensions
- **Container Platform**: Docker Desktop for local development
- **Version Control**: Git and GitHub for code management
- **Documentation**: Markdown editors and diagram tools

Continuous Learning Path

This intensive program provides a foundation for continued growth in:

- **Large Language Model Fine-tuning**
- **Advanced Agent Architectures**
- **MLOps and AI System Operations**
- **AI Ethics and Responsible AI Development**
- **Specialized Domain Applications**

This comprehensive learning plan is designed to provide hands-on experience with cutting-edge AI technologies while building production-ready systems. Each day builds upon previous learning while introducing new concepts and practical applications.